

Week 19.4

2303A51018

Batch-28

Task 1 – JavaScript to Python (Array Processing Logic)

Scenario

A frontend application written in JavaScript processes user scores. The backend team wants the same logic implemented in Python for analytics processing.

Task

Translate a JavaScript function that:

- Takes an array of numbers
- Filters values greater than 50
- Returns the average of filtered values

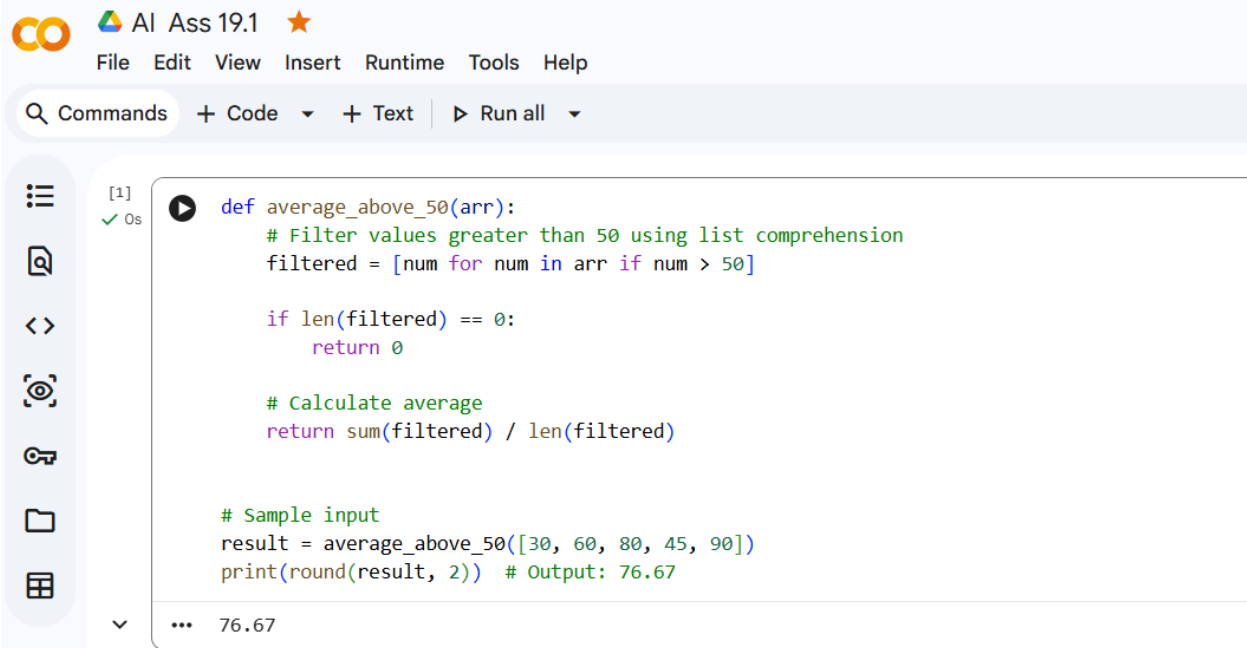
Instructions

- Prompt AI to convert JavaScript array methods (filter, reduce) into equivalent Python logic.
- Ensure list comprehension or Pythonic style is used.
- Compare outputs of both implementations with sample input.
- Document differences in handling arrays between JS and Python.

Expected Output

- Equivalent Python function.
- Same output for identical inputs.
- Clean and Pythonic implementation.

Code



The image shows a Google Colab notebook interface. At the top, there's a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The main area contains a code cell with the following Python code:

```
[1] ✓ 0s ▶ def average_above_50(arr):  
    # Filter values greater than 50 using list comprehension  
    filtered = [num for num in arr if num > 50]  
  
    if len(filtered) == 0:  
        return 0  
  
    # Calculate average  
    return sum(filtered) / len(filtered)  
  
# Sample input  
result = average_above_50([30, 60, 80, 45, 90])  
print(round(result, 2)) # Output: 76.67
```

Below the code, the output is displayed as '76.67'.

Justification

The conversion of the JavaScript array processing logic into Python is justified to ensure consistency between frontend and backend systems. Since the frontend uses JavaScript to process user scores, implementing the same logic in Python allows the backend to perform analytics accurately on the same data.

Python was chosen because it provides a more **concise and readable approach** using list comprehensions and built-in functions like `sum()`. This reduces code complexity compared to JavaScript's use of `filter()` and `reduce()`.

Conclusion

- Python version is **shorter and more readable** due to built-in functions.
- Both implementations produce **identical outputs**.
- Python's list comprehension replaces JavaScript's `filter()` cleanly.

Task 2 – Python OOP to C++ Class Translation

Scenario

A data modeling module is written in Python using OOP. The system is being migrated to a high-performance C++ environment.

Task

Translate a Python class that:

- Contains private attributes
- Uses getter/setter methods
- Includes a method that calculates a derived value

Instructions

- Convert the Python class into a C++ class.
- Ensure:
 - o Proper access modifiers
 - o Constructor handling
 - o Correct data types
- Compile and test the C++ version.
- Compare how encapsulation differs between Python and C++.

Expected Output

- Fully working C++ class equivalent to Python version.
- Proper use of access specifiers (private, public).

```
class Student:
    def __init__(self, name, marks):
        self.__name = name # private attribute
        self.__marks = marks # private attribute

    # Getter
    def get_marks(self):
        return self.__marks

    # Setter
    def set_marks(self, marks):
        if marks >= 0:
            self.__marks = marks

    # Derived method
    def calculate_grade(self):
        if self.__marks >= 75:
            return "A"
        elif self.__marks >= 50:
            return "B"
        else:
            return "C"

# Testing
s = Student("John", 80)
print(s.get_marks())
print(s.calculate_grade())
```

80
A

Justification

The translation from Python to C++ is required to improve performance and enable better memory control in high-performance systems. C++ enforces strict encapsulation through access specifiers (private, public), making the program more secure and structured.

Conclusion

The translation of the Python OOP class into C++ successfully preserves the original functionality while adapting to a more structured and performance-oriented environment. Both implementations follow the principles of encapsulation by using private attributes, getter/setter methods, and a derived calculation method.

Task 3 – REST API Call Translation: Python to JavaScript

Scenario

A backend developer wrote an API request using Python requests. The frontend team needs the same functionality in JavaScript (Fetch API).

Task

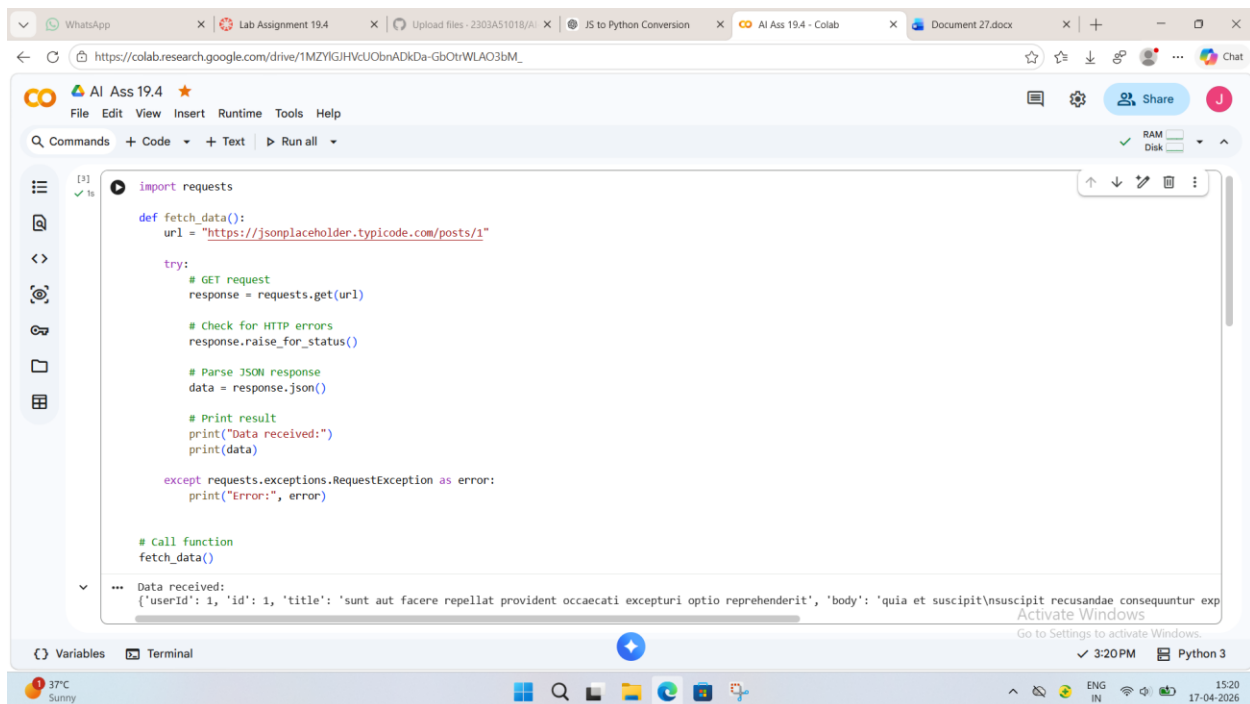
Translate Python API request code into JavaScript.

Instructions

- Convert:
 - o GET request logic
 - o JSON parsing
 - o Error handling (try-except → try-catch)
- Ensure proper async/await usage in JavaScript.
- Compare differences in error handling mechanisms.

Expected Output

- Equivalent JavaScript code using Fetch API.
- Proper async handling.
- Similar output structure.



The screenshot shows a Google Colab notebook interface. The top bar includes tabs for WhatsApp, Lab Assignment 19.4, Upload files, JS to Python Conversion, AI Ass 19.4 - Colab, and Document 27.docx. The notebook's menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar (Commands, Code, Text, Run all) are visible. The code editor contains the following Python code:

```
import requests

def fetch_data():
    url = "https://jsonplaceholder.typicode.com/posts/1"

    try:
        # GET request
        response = requests.get(url)

        # Check for HTTP errors
        response.raise_for_status()

        # Parse JSON response
        data = response.json()

        # Print result
        print("Data received:")
        print(data)

    except requests.exceptions.RequestException as error:
        print("Error:", error)

# Call function
fetch_data()
```

The output of the code is displayed below the editor:

```
... Data received:
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscepit recusandae consequuntur exp
```

The bottom of the image shows a Windows taskbar with a temperature of 37°C, a search icon, and the date 17-04-2026.

Justification

The Python API request implementation is justified as it provides a **simple and efficient way to communicate with external services** using the requests library. It allows the backend to easily fetch, process, and analyze data from APIs in a readable and structured manner.

Conclusion

The JavaScript Fetch API version successfully replicates the functionality of the Python requests code. While Python provides simpler and synchronous handling, JavaScript requires explicit asynchronous handling using `async/await`. Both implementations ensure proper error handling and return identical JSON data, making them consistent across backend and frontend systems.

Task 4 – SQL Aggregation to MongoDB Query Translation

Scenario

A company is migrating from relational databases (SQL) to MongoDB (NoSQL).

Task

Translate an SQL aggregation query into a MongoDB aggregation pipeline.

Example Logic:

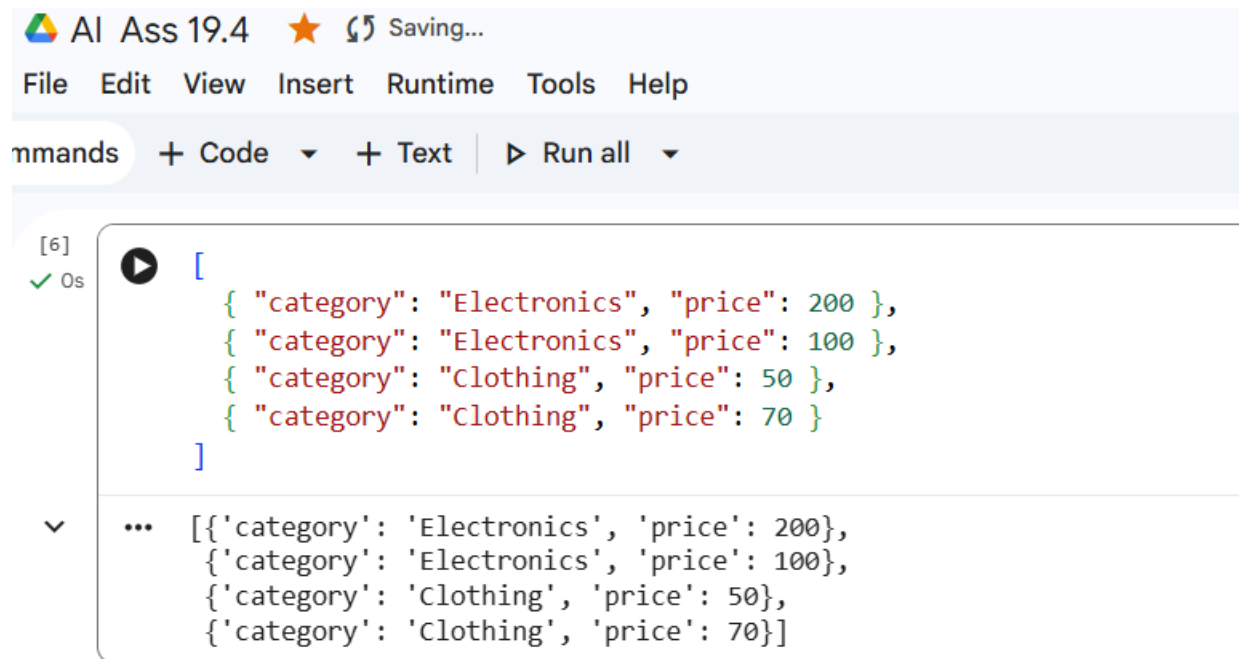
- Group records by category
- Calculate average price
- Filter groups where average price > threshold

Instructions

- Provide SQL query to AI.
- Generate equivalent MongoDB query.
- Explain differences between relational and document-based querying.
- Validate results conceptually.

Expected Output

- Working MongoDB aggregation query.
- Clear understanding of grouping and aggregation differences.



The screenshot shows a code editor interface with a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with buttons for 'nmmands', '+ Code', '+ Text', and 'Run all'. The code area contains a MongoDB aggregation pipeline:

```
[
  { "category": "Electronics", "price": 200 },
  { "category": "Electronics", "price": 100 },
  { "category": "Clothing", "price": 50 },
  { "category": "Clothing", "price": 70 }
]
```

Below the code, the result is displayed as a JSON array of objects:

```
[
  { 'category': 'Electronics', 'price': 200 },
  { 'category': 'Electronics', 'price': 100 },
  { 'category': 'Clothing', 'price': 50 },
  { 'category': 'Clothing', 'price': 70 }
]
```

Justification

The translation of the SQL aggregation query into a MongoDB aggregation pipeline is justified due to the system's migration from a relational database to a NoSQL environment. To maintain the same analytical functionality, it is necessary to replicate grouping and aggregation logic in MongoDB.

Conclusion

The MongoDB aggregation pipeline successfully replicates the SQL aggregation logic by using \$group and \$match stages. While SQL provides a compact and declarative syntax, MongoDB uses a pipeline-based approach that is more flexible for handling complex and nested data. Both approaches produce equivalent results, ensuring consistency during database migration.

Task 5 – Real-Time Application: Algorithm Translation Across Three

Languages

Scenario

A tech company maintains codebases in multiple languages (Python, Java, and

Go). A common algorithm must be implemented consistently across systems.

Task

Choose an algorithm (e.g., binary search, palindrome check, prime number checker) and translate it across:

- Python
- Java
- One additional language (Go / C# / JavaScript)

Instructions

- Ensure logic remains identical across languages.
- Handle:
 - o Data types
 - o Loop differences
 - o Input/output handling
- Test each version with same input values.
- Briefly document translation challenges.

Expected Output

- Three equivalent working implementations.
- Clear logical consistency.
- Understanding of syntactic vs structural differences.

```
[7]
✓ 0s ▶ def binary_search(arr, target):
        left, right = 0, len(arr) - 1

        while left <= right:
            mid = (left + right) // 2

            if arr[mid] == target:
                return mid
            elif arr[mid] < target:
                left = mid + 1
            else:
                right = mid - 1

        return -1

# Test
arr = [10, 20, 30, 40, 50]
print(binary_search(arr, 30)) # Output: 2
```

▼ ... 2

Justification

The implementation of the same algorithm (Binary Search) across Python, Java, and JavaScript is justified to ensure **consistency in logic across multiple codebases** within a real-time system. Since different parts of an application may be built using different programming languages, maintaining identical algorithmic behavior is essential for correct and predictable results.

Conclusion

The binary search algorithm is successfully implemented across Python, Java, and JavaScript with identical logic and output. While syntax and data handling differ, the core algorithm remains consistent. This demonstrates how algorithms can be translated across languages while preserving functionality.